Web 2 – Séance 09

Assets, CSS et MUI

Assets : fichiers statiques

- **Asset** → fichier qui n'est pas du code : image, police, icône, vidéo…
- Les assets sont **servis par le serveur** et utilisés par le navigateur
- Le navigateur peut **mettre en cache** les assets pour ne pas les re-télécharger

Emplacements des assets dans un projet Vite

- **Dossier** `public/` : fichiers copiés tels quels, servis à la racine du site
  - `public/logo.png` → `<img src="/logo.png" />`
  - Le navigateur peut garder l'ancien fichier en cache si on remplace le fichier sans changer son nom
  - **Utile pour** les fichiers qui doivent garder leur nom exact (favicon, robots.txt) ou ne changent jamais
- **Dossier** `src/assets/` : fichiers **importés** dans le code, comme un module
  - Vite vérifie que le fichier existe au build (erreur si le chemin est faux)
  - Vite ajoute un hash au nom (`logo-4f3a2b.png`)
  - Le hash change si le fichier change, donc le navigateur télécharge la nouvelle version
  - **Utile pour** les images qui font partie de l'interface et que l'on modifie au fil du développement (logo, illustrations, icônes)
- Les **données** qui vont changer au fil du temps (e.g. photos des recettes) ne peuvent pas être connues durant le développement. Elles sont gérées par un autre serveur (backend) et récupérées via des URL.

Utiliser des assets dans un projet React

- Dossier `public/` : pas d'import, chemin depuis la racine du site

```
<img src="/logo.png" alt="Logo MiamMiam" />
```

- Dossier `src/assets/` : import depuis le code, chemin relatif au fichier

```
import logo from "../assets/logo.png";

const Header = () => {
  return (
    <header>
      <img src={logo} alt="Logo MiamMiam" />
      <h1>MiamMiam</h1>
    </header>
  );
};
```

- Asset externe : URL complète

```
<img src="https://images.unsplash.com/photo-... " alt="Guacamole" />
```

CSS dans un projet React

Un fichier CSS est importé dans un fichier TypeScript : Vite l'injecte dans la page.

```
// main.tsx (généré par Vite)
import "./index.css";

// RecipeCard.tsx
import "./RecipeCard.css";
```

⚠️ **CSS global**, même importé depuis un composant spécifique :

```
/* RecipeCard.css */
.title {
  color: #e65100;
}
```

- La règle `.title` s'applique à **tous** les éléments `className="title"` de l'application
- Deux composants qui utilisent le même nom de classe entrent en conflit
- Le fichier n'est « rattaché » au composant que par convention

CSS Modules : des classes locales

Un fichier nommé `*.module.css` est un **CSS Module** : Vite renomme chaque classe pour la rendre unique.

```css
/* RecipeCard.module.css */
.card {
  border: 1px solid #ddd;
  border-radius: 8px;
  padding: 16px;
}
.title {
  color: #e65100;
}
```

```tsx
import styles from "./RecipeCard.module.css";
const RecipeCard = ({ title }: RecipeCardProps) => {
  return (
    <div className={styles.card}>
      <h2 className={styles.title}>{title}</h2>
    </div>
  );
};
```

- `styles.title` vaut par exemple `"_title_1×7a2_5"`
- `styles` est un objet dont les clés sont les noms de classes du fichier

Style inline

L'attribut `style` reçoit un **objet** TypeScript, pas une chaîne.

```
<div style={{ backgroundColor: "#fff3e0", padding: 16, borderRadius: "8px" }}>
  Contenu
</div>
```

- Doubles accolades : les premières pour l'expression JSX, les secondes pour l'objet
- Propriétés en **camelCase** : `background-color` → `backgroundColor`
- Un nombre est interprété en pixels : `padding: 16` = `padding: "16px"`
- Peut rendre le code peu lisible si on met beaucoup de styles inline
- À réserver aux styles **calculés** (ex : couleur qui dépend d'une prop), pas à la mise en forme générale

```
<span style={{ color: difficulty > 3 ? "red" : "green" }}>{difficulty}/5</span>
```

Bibliothèque de composants

Une **bibliothèque de composants** fournit des composants React déjà stylés :

- Boutons, cartes, champs de formulaire, menus, icônes…
- Un **design system** : couleurs, typographie, espacements cohérents
- Accessibilité (clavier, lecteurs d'écran) prise en charge
- Personnalisable via un **thème**

Exemples : MUI, Chakra UI, Ant Design, shadcn/ui…

Dans ce cours : **MUI** (Material UI), qui implémente le Material Design de Google.

Installer MUI

```
npm install @mui/material @emotion/react @emotion/styled @mui/icons-material
```

- `@mui/material` : les composants
- `@emotion/react`, `@emotion/styled` : la librairie CSS-in-JS utilisée par MUI pour générer les styles
- `@mui/icons-material` : plus de 2000 icônes Material sous forme de composants

```
import { Button } from "@mui/material";
import AddIcon from "@mui/icons-material/Add";

const AddButton = () => (
  <Button variant="contained" startIcon={<AddIcon />}>
    Ajouter une recette
  </Button>
);
```

Chaque composant MUI est un composant React : il se configure **avec des props**.

Thème et CssBaseline

Le **thème** définit les couleurs, polices et espacements de toute l'application.

```tsx
// App.tsx
import { createTheme, CssBaseline, ThemeProvider } from "@mui/material";

const theme = createTheme({
  palette: {
    primary: { main: "#e65100" }, // orange MiamMiam
  },
});

const App = () => {
  return (
    <ThemeProvider theme={theme}>
      <CssBaseline />
      {/* ... le reste de l'application ... */}
    </ThemeProvider>
  );
};
```

- `ThemeProvider` rend le thème disponible à tous les composants MUI qu'il contient (`children`)
- `CssBaseline` remplace le CSS par défaut du navigateur par une base cohérente (marges, police)

La prop sx

Tous les composants MUI acceptent une prop `sx` pour les styles ponctuels.

```
import { Box } from "@mui/material";

<Box sx={{ p: 2, mt: 4, bgcolor: "primary.main", color: "white", borderRadius: 2 }}>
  Contenu
</Box>
```

- Comme `style`, mais avec accès au **thème**
  - `bgcolor: "primary.main"` : couleur du thème
  - `p: 2` : padding de 2 unités d'espacement (1 unité = 8px → 16px)
  - `mt`, `mb`, `mx`, `py` … : raccourcis de margin et padding, `mt` = margin-top
- `Box` = un `<div>` qui accepte `sx`
- Préférer `sx` au `style` inline dans une application MUI

Typography

`Typography` affiche du texte avec les styles du thème.

```
import { Typography } from "@mui/material";

<Typography variant="h4" component="h1">MiamMiam</Typography>
<Typography variant="h6" component="h2">Pâtes Carbonara</Typography>
<Typography variant="body1">Un classique italien.</Typography>
<Typography variant="body2" color="text.secondary">25 min</Typography>
```

- `variant` : l'**apparence** (taille, graisse) – `h1` à `h6`, `body1`, `body2`, `caption`…
- `component` : la **balise HTML** générée
- Les séparer permet d'avoir un titre `<h1>` (sémantique, accessibilité) qui n'est pas énorme
- `color="text.secondary"` : gris du thème pour les informations secondaires

Card

```tsx
import { Card, CardActions, CardContent, CardMedia, Button, Typography } from "@mui/material";

const RecipeCard = ({ title, imageUrl, duration }: RecipeCardProps) => (
  <Card sx={{ maxWidth: 345 }}>
    <CardMedia component="img" height="200" image={imageUrl} alt={title} />
    <CardContent>
      <Typography variant="h6" component="h2">
        {title}
      </Typography>
      <Typography variant="body2" color="text.secondary">
        {duration} min
      </Typography>
    </CardContent>
    <CardActions>
      <Button size="small">Voir la recette</Button>
    </CardActions>
  </Card>
);
```

- `CardMedia` : image (ici une `<img>`, grâce à `component="img"`)
- `CardContent` : le contenu textuel, avec les marges intérieures du thème
- `CardActions` : la zone des boutons

Chip et Rating

```
import { Chip, Rating } from "@mui/material";

<Chip label="Plat" color="primary" size="small" />
<Chip label="Végétarien" variant="outlined" size="small" />

<Rating value={3} max={5} readOnly />
```

- `Chip` : petite étiquette, idéale pour une catégorie ou un tag
- `Rating` : note en étoiles
  - `readOnly` : affichage seul, l'utilisateur ne peut pas cliquer
  - Parfait pour afficher la difficulté d'une recette (1 à 5)
  - Sans `readOnly`, le composant est modifiable : on verra comment récupérer la valeur choisie avec l'état (séance 11)

Mise en page : Stack et Container

```
import { Container, Stack } from "@mui/material";

<Container maxWidth="lg">
  <Stack direction="row" spacing={2}>
    <RecipeCard title="Pâtes Carbonara" imageUrl="..." duration={25} difficulty={2} />
    <RecipeCard title="Guacamole" imageUrl="..." duration={10} difficulty={1} />
  </Stack>
</Container>
```

- `Container` : centre le contenu avec une largeur maximale (`sm` , `md` , `lg` , `xl`)
- `Stack` : aligne ses enfants en ligne (`row`) ou en colonne (`column` , par défaut)
  - `direction="row"` ou `direction="column"`
  - `spacing={2}` : espace de 2 unités (16px) entre les enfants

Grid

```
import { Grid } from "@mui/material";
<Grid container spacing={2}>
  <Grid size={{ xs: 12, sm: 6, md: 4 }}>
    <RecipeCard title="Pâtes Carbonara" imageUrl="..." duration={25} difficulty={2} />
  </Grid>
  <Grid size={{ xs: 12, sm: 6, md: 4 }}>
    <RecipeCard title="Guacamole" imageUrl="..." duration={10} difficulty={1} />
  </Grid>
  <Grid size={{ xs: 12, sm: 6, md: 4 }}>
    <RecipeCard title="Mousse au chocolat" imageUrl="..." duration={20} difficulty={3} />
  </Grid>
</Grid>
```

- `Grid container` : conteneur de la grille, un `Grid` sans `container` est un élément de la grille
- La grille est divisée en **12 colonnes**
- `size` : nombre de colonnes occupées selon la largeur de l'écran
  - `size={6}` : 6 colonnes quelle que soit la taille de l'écran
  - `xs` (mobile) : 12 → une carte par ligne
  - `sm` (tablette) : 6 → deux cartes par ligne

Lire la documentation MUI

La documentation https://mui.com/material-ui/ est la référence :

- **Démos** : chaque composant a des exemples interactifs, avec le code
- **API** : liste complète des props de chaque composant, avec leur type
- Toujours vérifier la **version** : les exemples trouvés en ligne utilisent souvent une ancienne API (`Grid item xs={6}` , `<Stack alignItems="center">` …) qui ne fonctionne plus avec MUI v9

Méthode pour intégrer un composant :

1. Trouver le composant dans la liste de la documentation
2. Copier la démo la plus proche de votre besoin
3. Adapter les props à vos données
4. Consulter l'onglet API pour les options manquantes

Récapitulatif Séance 09

- **Assets** – `public/` pour les fichiers servis tels quels, `src/assets/` pour les fichiers importés
- **Import d'image** – L'import retourne l'URL du fichier
- **CSS global** – Tout CSS importé s'applique à toute l'application
- **CSS Modules** – `*.module.css`, classes renommées et locales au composant
- **Style inline** – Objet en camelCase, pour les styles calculés
- **MUI** – Bibliothèque de composants React stylés, configurés par des props
- **Thème** – `createTheme`, `ThemeProvider`, `CssBaseline`
- **sx** – Styles ponctuels avec accès au thème
- **Composants** – `Typography`, `Card`, `Chip`, `Rating`, `Button`, `Box`, `Stack`, `Container`, `Grid`

Prochaine séance : Séance 10 – Modules et collections

Exercice filé S09

1. Téléchargez l'image d'accueil `miammiam.png` et placez-la dans `src/assets/`
2. Dans `App.tsx`, importez l'image et affichez-la dans un `<img>` au-dessus du titre
3. Supprimez le contenu de `src/index.css` et le fichier `src/App.css` (et son import dans `App.tsx`)
4. Installez MUI et les icônes MUI
5. Dans `App.tsx`, créez un thème avec l'orange `#e65100` comme couleur principale, et ajoutez `ThemeProvider` et `CssBaseline`
6. Réécrivez `RecipeCard` avec `Card`, `CardMedia`, `CardContent`, `CardActions` et `Typography`
7. Affichez la difficulté avec un `Rating` en lecture seule, et ajoutez une prop `category` affichée dans un `Chip`
8. Réécrivez `PageLayout` avec une `AppBar` et une `Toolbar` pour l'en-tête, et un `Container` pour le contenu
9. Dans `RecipeList`, disposez les cartes avec un `Stack`
10. **Optionnel** : utilisez `Grid` pour disposer les cartes sur plusieurs colonnes selon la taille de l'écran